

WEEK-4 : CUSTOMER AND PRODUCT ORDER ANALYSIS

1. Scenario

Art Work Shop manages customer orders and product sales using a database system. The database stores customer, product and order details. Reports are generated to view customer order history, product-wise order quantities, and customer purchase details. These reports help in analyzing workshop orders and customer purchases.

2. Create Table – Orders

The Orders table stores order ID, customer ID, order date, total amount and order status. Primary key, foreign key, NOT NULL and CHECK constraints are used to maintain data integrity.

```
SQL> CREATE TABLE Orders (Order_ID NUMBER PRIMARY KEY, Customer_ID VARCHAR2(20) NOT NULL, Order_Date DATE NOT NULL, Total_Amount NUMBER(10,2) NOT NULL, Order_Status VARCHAR2(20) NOT NULL, CONSTRAINT fk_orders_customer FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID), CONSTRAINT chk_order_status CHECK (Order_Status IN ('Pending','Shipped','Delivered')));  
Table created.
```

3. Create Table – Order_Details

The Order_Details table stores order detail ID, order ID, product ID, quantity and price. Primary key, foreign key, NOT NULL and CHECK constraints are used.

```
SQL> CREATE TABLE Order_Details (Order_Detail_ID NUMBER PRIMARY KEY, Order_ID NUMBER NOT NULL, Product_ID VARCHAR2(7) NOT NULL, Quantity NUMBER NOT NULL, Price NUMBER(10,2) NOT NULL, CONSTRAINT fk_od_order FOREIGN KEY(Order_ID) REFERENCES Orders(Order_ID), CONSTRAINT fk_od_product FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID), CONSTRAINT chk_od_quantity CHECK(Quantity > 0), CONSTRAINT chk_od_price CHECK(Price > 0));  
Table created.
```

4. Relationships

The following relationships are maintained between the tables:

- CUSTOMER_ID in Customer → CUSTOMER_ID in Orders
- ORDER_ID in Orders → ORDER_ID in Order_Details
- PRODUCT_ID in Product → PRODUCT_ID in Order_Details

5. Constraints

The following constraints are used in the database:

- Primary Key
- Foreign Key
- NOT NULL
- CHECK Constraint

6. Insert Records

Customer, Category, Product, Orders and Order_Details records were inserted into the database using SQL INSERT statements.

7. Update Record

An existing order record was updated to modify the order information.

```
SQL> UPDATE Orders
  2  SET Order_Status = 'Shipped'
  3  WHERE Order_ID = 1006;

1 row updated.
```

8. Delete Record

An existing order record and its corresponding order detail were deleted from the database.

```
SQL> DELETE FROM Order_Details
  2  WHERE Order_ID = 1010;

1 row deleted.

SQL> DELETE FROM Orders
  2  WHERE Order_ID = 1010;

1 row deleted.
```

9. Report 1 – Customer Order History

This report displays the customer name, order ID, order date, total amount and order status.

```
SQL> SELECT FIRST_NAME, LAST_NAME, ORDER_ID, ORDER_DATE, TOTAL_AMOUNT
  2  FROM Customer
  3  JOIN Orders
  4  ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID;
```

FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
Beema	ponting	1001	20-SEP-26	67500
priya	sakthi	1002	21-SEP-26	45000
Arun	kumar	1003	22-SEP-26	3400
Anu	Priya	1004	23-SEP-26	75000
Reena	das	1005	24-SEP-26	7500

10. Report 2 – Product-wise Order Report

This report displays the total quantity ordered for each product.

```
SQL> SELECT p.Product_Name,  
2      SUM(od.Quantity) AS Total_Quantity_Ordered  
3 FROM Product p  
4 JOIN Order_Details od  
5 ON p.Product_ID = od.Product_ID  
6 GROUP BY p.Product_Name;
```

PRODUCT_NAME

TOTAL_QUANTITY_ORDERED

Gaming Headset	2
----------------	---

Gaming Controller	6
-------------------	---

Mechanical Keyboard	2
---------------------	---

PRODUCT_NAME

TOTAL_QUANTITY_ORDERED

Gaming Monitor	1
----------------	---

Bluetooth Speaker	1
-------------------	---

Wireless Mouse	2
----------------	---

6 rows selected.

11. Report 3 – Customer Purchase Analysis

This report analyzes customer purchases by identifying the number of orders, total spending and average order value of customers.

- Customers with Maximum Orders

Identifies the customers who have placed the maximum number of orders.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, COUNT(Orders.ORDER_ID) AS TOTAL_ORDERS FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME HAVING COUNT(Orders.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(ORDER_ID) AS ORDER_COUNT FROM Orders GROUP BY CUSTOMER_ID));
```

FIRST_NAME	LAST_NAME	TOTAL_ORDERS
priya	sakthi	2
Arun	kumar	2
Anu	Priya	2
Beema	ponting	2

- Total Spending by Each Customer

Calculates the total amount spent by each customer.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, SUM(Orders.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME;
```

FIRST_NAME	LAST_NAME	TOTAL_SPENDING
priya	sakthi	112500
Arun	kumar	48400
Anu	Priya	78400
Reena	das	7500
Beema	ponting	125000

- Average Order Value

Calculates the average order value for each customer.

```
SQL> SELECT Customer.FIRST_NAME, Customer.LAST_NAME, AVG(Orders.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM Customer JOIN Orders ON Customer.CUSTOMER_ID = Orders.CUSTOMER_ID GROUP BY Customer.CUSTOMER_ID, Customer.FIRST_NAME, Customer.LAST_NAME;
```

FIRST_NAME	LAST_NAME	AVERAGE_ORDER_VALUE
priya	sakthi	56250
Arun	kumar	24200
Anu	Priya	39200
Reena	das	7500
Beema	ponting	62500